

REKAYASA PERANGKAT LUNAK

Modul Pembelajaran: Model Proses Perangkat Lunak (SDLC)

Penjelasan komprehensif mengenai siklus hidup pengembangan perangkat lunak, aktivitas dasar, dan ragam model arsitektur seperti Waterfall, Prototype, RAD, Incremental, hingga Spiral.

Contents

1	Pengantar Siklus Hidup Perangkat Lunak	2
1.1	Konsep Model Proses (Paradigma)	2
1.2	Memahami SDLC (Software Development Life Cycle)	2
2	1. Model Sekuensial Linier (Waterfall)	4
2.1	Evaluasi Model Waterfall	4
3	2. Model Prototipe (Prototyping)	5
3.1	Evaluasi Model Prototipe	5
4	3. Rapid Application Development (RAD)	6
4.1	Evaluasi Model RAD	6
5	4. Model Evolusioner Inkremental	8
5.1	Evaluasi Model Inkremental	8
6	5. Model Evolusioner Spiral	9
6.1	Evaluasi Model Spiral	9

1 Pengantar Siklus Hidup Perangkat Lunak

Membangun sebuah perangkat lunak berskala besar ibarat membangun sebuah gedung pencakar langit. Proses ini tidak bisa sekadar diawali dengan niat lalu langsung menyusun batu bata (menulis kode). Proses rekayasa perangkat lunak melibatkan keseimbangan antara Proses Manajemen Proyek dan Proses Pengembangan Perangkat Lunak itu sendiri.

1.1 Konsep Model Proses (Paradigma)

Menurut Ian Sommerville, sebuah model proses atau paradigma rekayasa perangkat lunak menunjukkan suatu strategi untuk melindungi dan mengelola proses, metode, serta alat bantu secara efektif agar dapat mengirimkan produk perangkat lunak dengan sukses.

Bagaimana cara memilih model proses yang tepat? Pemilihannya didasarkan pada tiga hal utama:

- Proyek dan aplikasi yang alami (karakteristik dasar dari aplikasi yang akan dibuat).
- Metode dan alat bantu yang digunakan oleh tim.
- Kebutuhan akan kontrol dan pengiriman (*delivery*) produk.

Terlepas dari model apa yang nantinya dipilih, terdapat empat aktivitas dasar yang pasti ada di setiap proses rekayasa perangkat lunak:

1. **Software specification** (Spesifikasi perangkat lunak).
2. **Software development** (Pengembangan perangkat lunak).
3. **Software validation** (Validasi atau pengujian perangkat lunak).
4. **Software evolution** (Evolusi atau pemeliharaan perangkat lunak).

1.2 Memahami SDLC (Software Development Life Cycle)

Software Development Life Cycle (SDLC) adalah sebuah pendekatan yang disiplin dan sistematis dalam membagi proses pengembangan perangkat lunak ke dalam berbagai fase yang terstruktur, seperti fase kebutuhan (*requirement*), perancangan (*design*), dan pengkodean (*coding*).

Tujuan utama dari penerapan fase-fase SDLC ini adalah untuk membantu manajer dan pengembang melacak jadwal (*schedule*), biaya (*cost*), dan kualitas dari proyek perangkat lunak tersebut.

Secara rinci, terdapat enam fase utama dalam sebuah SDLC:

1. **Feasibility analysis phase:** Meliputi analisis kelayakan dan kebutuhan proyek.
2. **Requirement analysis and specification phase:** Meliputi proses pengumpulan (*gathering*), menganalisis, memvalidasi, dan menspesifikasikan kebutuhan sistem ke dalam dokumen SRS (*Software Requirement Specification*).
3. **Design phase:** Meliputi proses penerjemahan kebutuhan yang dispesifikasikan di SRS menjadi struktur logis yang nantinya dapat diimplementasikan ke dalam bahasa pemrograman.

4. **Coding phase:** Meliputi implementasi desain (dari dokumen desain) menjadi kode eksekusi menggunakan bahasa pemrograman tertentu.
5. **Testing phase:** Meliputi pencarian dan pendeteksian *error* (kesalahan) di dalam perangkat lunak.
6. **Maintenance phase:** Meliputi penerapan perubahan dan penambahan kebutuhan baru pada perangkat lunak di lokasi pengguna/pelanggan (*customer location*).

2 1. Model Sekuensial Linier (Waterfall)

Model proses perangkat lunak yang paling klasik dan paling mudah dipahami adalah Model **Waterfall** (Air Terjun). Disebut air terjun karena aliran prosesnya mengalir ke bawah secara linier dan berurutan dari satu tahap ke tahap selanjutnya.

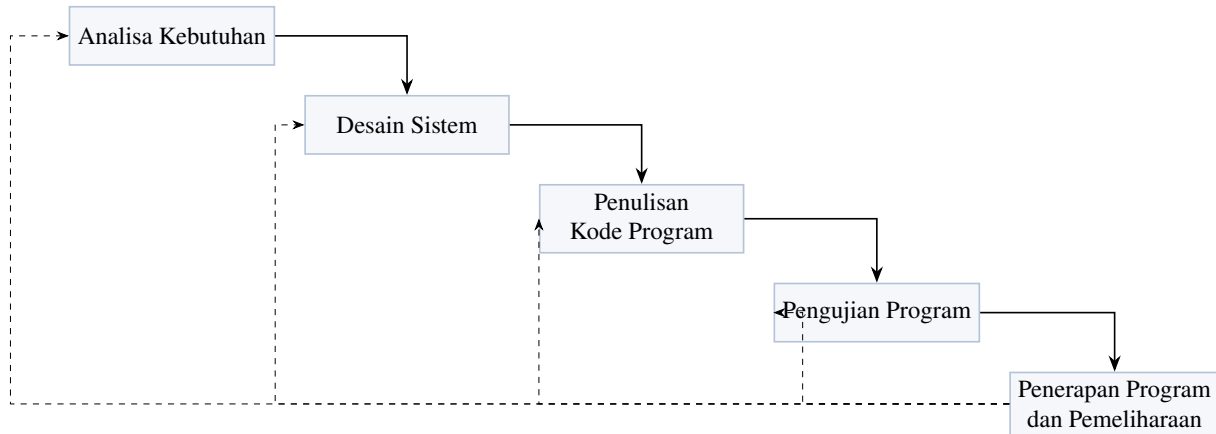


Figure 1: Skema Aliran Model Waterfall.

2.1 Evaluasi Model Waterfall

Model ini mengunci setiap fase sebelum lanjut ke fase berikutnya. Berikut adalah karakteristik keunggulannya:

- **Kualitas Tinggi:** Kualitas dari sistem yang dihasilkan akan sangat baik karena setiap langkah diuji dan diselesaikan secara tuntas.
- **Dokumentasi Terorganisir:** Dokumen pengembangan sistem menjadi sangat rapi dan terorganisir.
- **Kepastian Kebutuhan:** Sangat cocok digunakan jika kebutuhan aplikasi sudah diketahui dengan baik dari awal proyek.

Namun, karena sifatnya yang kaku, model ini memiliki beberapa kelemahan fatal:

- **Efek Domino Kesalahan:** Kesalahan kecil di awal akan menjadi masalah yang sangat besar jika tidak segera diketahui sejak awal.
- **Kendala Pelanggan:** Pelanggan seringkali sulit untuk menyatakan dan membayangkan kebutuhannya secara eksplisit di awal proyek.
- **Waktu Tidak Efisien:** Proses pengembangan harus menunggu tahap sebelumnya selesai baru bisa lanjut ke tahap berikutnya, sehingga banyak waktu tunggu (*idle*).

3 2. Model Prototipe (Prototyping)

Menjawab kelemahan Waterfall di mana pelanggan sulit membayangkan kebutuhan di awal, hadirlah Model Prototipe. Model ini berfungsi sebagai mekanisme untuk mengidentifikasi kebutuhan perangkat lunak di awal dengan cepat. Melalui "Perancangan Kilat", model ini menyajikan gambaran aspek perangkat lunak yang kemudian digunakan untuk menyaring dan memvalidasi kebutuhan pelanggan.

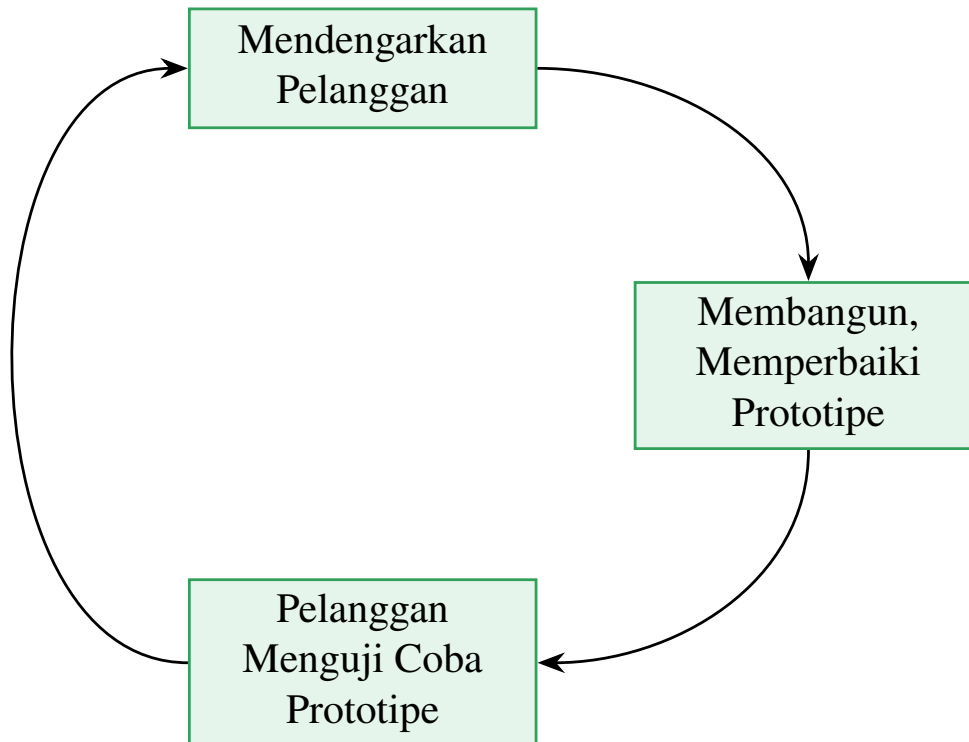


Figure 2: Siklus Berulang pada Model Prototipe.

3.1 Evaluasi Model Prototipe

Keunggulan dari model Prototipe adalah pendekatan interaktifnya:

- Terdapat **komunikasi yang baik** antara pengembang dan pelanggan.
- Pengembang dapat bekerja lebih baik dan tepat sasaran.
- Pelanggan **berperan sangat aktif** selama proses pengembangan sistem.

Namun, model ini juga menyimpan jebakan (risiko) yang sering membuat kualitas akhir perangkat lunak menurun:

- Pelanggan terkadang tidak paham bahwa prototipe perangkat lunak yang ada belum mencantumkan standar kualitas dan pemeliharaan untuk jangka waktu yang lama.
- Pengembang biasanya ingin cepat-cepat menyelesaikan proyek, sehingga mereka menggunakan algoritma dan bahasa pemrograman yang sederhana agar lebih cepat selesai. Padahal, mereka melakukan itu tanpa memikirkan bahwa program prototipe tersebut akan menjadi *blue print* sistem ke depannya.

4 3. Rapid Application Development (RAD)

Model **Rapid Application Development (RAD)** adalah model yang sangat mengedepankan kecepatan. RAD menekankan siklus pengembangan yang sangat pendek, biasanya ditekan hingga selesai dalam **60-90 hari** saja! Pengembangan yang sangat cepat ini dicapai melalui pendekatan konstruksi berbasis komponen, di mana beberapa tim dipecah secara paralel.

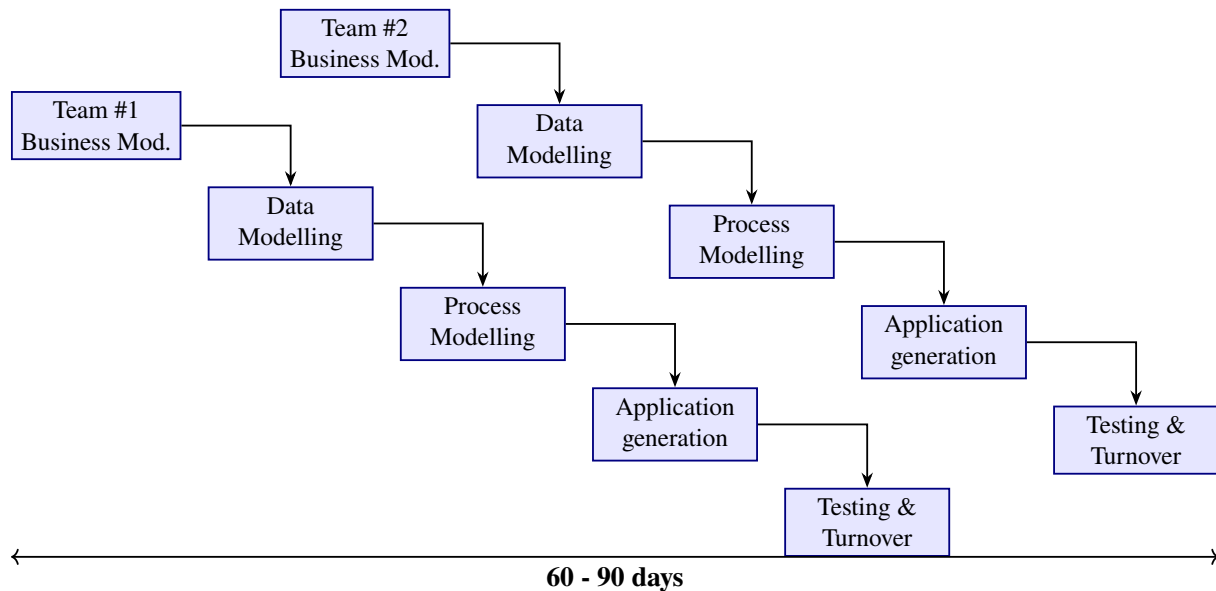


Figure 3: Skema Tim Paralel pada Model RAD.

4.1 Evaluasi Model RAD

Karena sifatnya yang cepat, RAD memiliki keunggulan tersendiri:

- Lebih fleksibel karena pengembang dapat melakukan proses desain ulang pada saat yang bersamaan.
- Bisa mengurangi penulisan kode yang kompleks dengan menggunakan komponen siap pakai.
- Tampilan yang dihasilkan lebih standar dan nyaman berkat bantuan *software-software* pendukung dalam pembuatannya.

Namun, risiko dari memaksakan kecepatan adalah kualitas yang sering dikorbankan:

- Lebih banyak terjadi kesalahan apabila tim hanya mengutamakan kecepatan pembuatan.
- Fasilitas-fasilitas banyak yang terpaksa dikurangi karena terbatasnya waktu yang tersedia.
- Sistem yang dihasilkan menjadi sulit untuk diaplikasikan di tempat yang lain.
- Fasilitas yang sebenarnya tidak perlu terkadang harus disertakan secara paksa, karena tim menggunakan komponen yang sudah jadi (*drag-and-drop*).

Modifikasi Modern dari RAD: Agile Software

Konsep kecepatan RAD berevolusi menjadi pendekatan **Agile Software**. Agile menekankan interaksi pengembang dengan pelanggan yang bertujuan membangun perangkat lunak yang tangkas dalam menghadapi perubahan. Beberapa macam turunan Agile antara lain:

1. **Scrum:** Membagi pekerjaan ke dalam tim-tim yang saling bekerja secara *overlap* (tumpang tindih).
2. **Extreme Programming (XP):** Mengutamakan intensitas komunikasi yang sangat sering antara pengembang dan pelanggan.

5 4. Model Evolusioner Inkremental

Model **Inkremental** adalah perpaduan jenius yang menggabungkan elemen-elemen dari model sekuensial linier (Waterfall) yang diimplementasikan secara berulang, dengan filosofi dari model *prototype* iteratif. Setiap iterasi akan menghasilkan sebuah "Inkremen" (potongan modul aplikasi yang bisa langsung berfungsi).

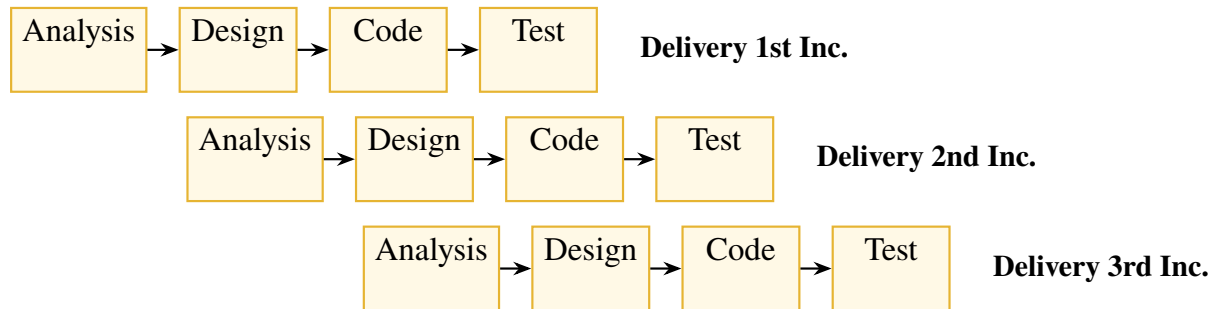


Figure 4: Pengembangan Bertahap pada Model Inkremental.

5.1 Evaluasi Model Inkremental

Pendekatan cicilan fungsionalitas ini memberi nilai lebih:

- Pelanggan **tidak perlu menunggu sampai seluruh sistem selesai** dikirimkan untuk mulai mengambil keuntungan dari fungsionalitas sistem tersebut.
- **Risiko untuk kegagalan proyek secara keseluruhan lebih rendah**, karena setiap modul dikembangkan dan dievaluasi secara berkala.

Namun, memecah sistem juga memiliki tantangan tersendiri:

- Inkremen yang dibuat harus relatif lebih kecil (patokannya tidak boleh lebih dari 20.000 baris kode).
- Setiap inkremen harus dipastikan bisa menyediakan sebagian dari fungsional utuh sistem.
- Terdapat kesulitan untuk memetakan spesifikasi persyaratan pelanggan ke dalam potongan-potongan inkremen dengan ukuran yang benar-benar pas.

6 5. Model Evolusioner Spiral

Model terakhir dan yang paling tangguh untuk proyek raksasa adalah **Model Evolusioner Spiral**. Model ini merangkai dua sifat sekaligus: pendekatan metode iteratif (berulang) dengan kontrol yang ketat dan aspek sistematis khas dari model sekuensial linear (Waterfall).

Proses bergerak secara spiral dari pusat ke luar, melewati empat kuadran aktivitas secara berulang:

1. **Perencanaan (Komunikasi Pelanggan):** Memahami tujuan, alternatif, dan batasan (*Determine objectives, alternatives and constraints*).
2. **Analisis Risiko:** Mengevaluasi setiap alternatif yang ada, serta mengidentifikasi dan menyelesaikan risiko yang terdeteksi (*Evaluate alternatives, identify, resolve risks*).
3. **Rekayasa (Konstruksi):** Melakukan tahapan desain, pengkodean, dan pengujian secara sistematis (*Design, Code, Test*).
4. **Evaluasi Pelanggan (Peluncuran):** Melibatkan evaluasi dari pelanggan dan merencanakan fase putaran spiral berikutnya (*Review, Plan next phase*).

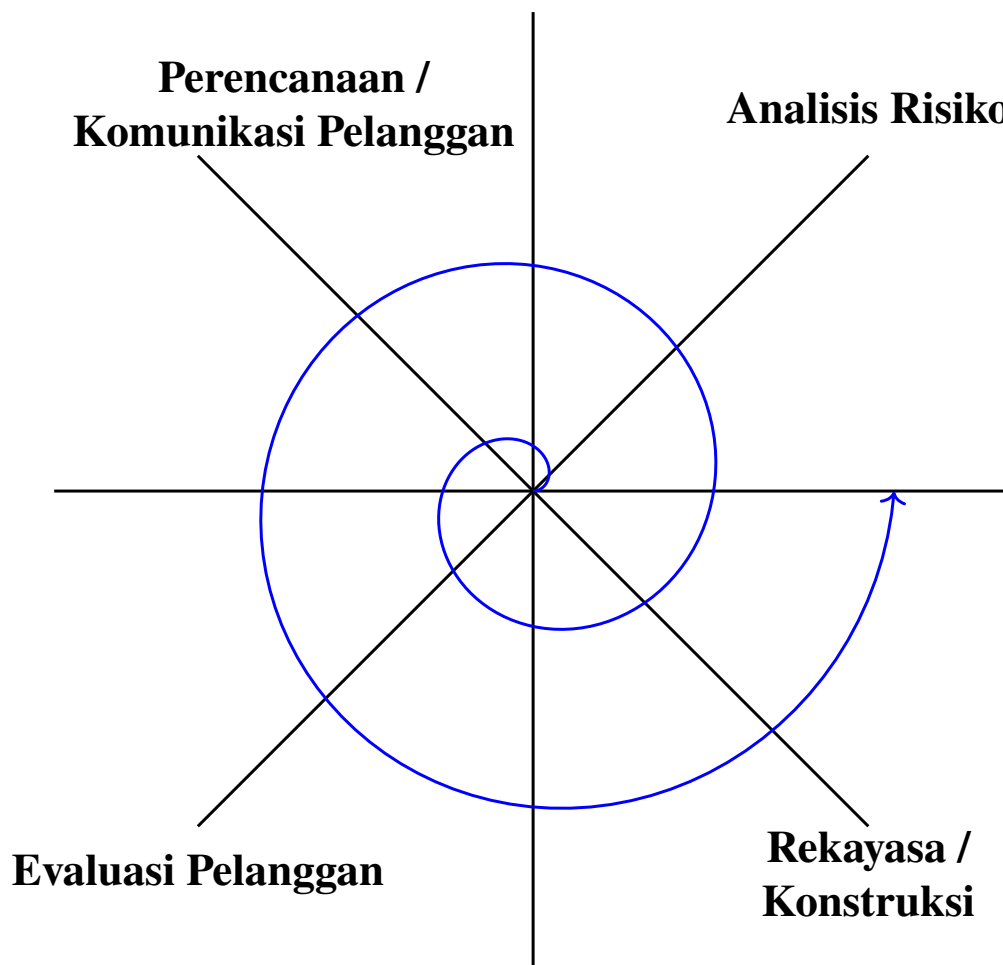


Figure 5: Representasi 4 Kuadran pada Model Spiral.

6.1 Evaluasi Model Spiral

Keunggulan utama dari model Spiral terletak pada manajemen risikonya:

- Model ini sangat baik dan disarankan untuk digunakan pada sistem dan *software* berskala besar.
- Adanya fase Analisa Risiko pada mekanisme ini dapat memperkecil risiko kegagalan yang fatal.
- Adanya kegiatan *prototyping* di dalam putaran spiral akan sangat memudahkan komunikasi teknis dengan konsumen.

Sebagai penyeimbang, kelemahan dari model super aman ini adalah:

- Memerlukan **waktu yang cukup lama** (dan biaya besar) untuk mengembangkan *software* karena harus melewati putaran evaluasi dan analisis risiko berulang kali.
- Sistem pengendaliannya dinilai kurang baik (karena kompleksitas perputaran iteratif yang harus ditangani manajer proyek).